Shree Shankar Narayan Education Trust's
PRAVIN ROHIDAS PATIL COLLEGE OF
ENGINEERING AND TECHNOLOGY
(Affiliated to the University of Mumbai,Recognised by DTE Govt.of Maharashtra & Approved by AICTE,New Delhi)
DTE CODE - 3277

Pravin Rohidas Patil College Of Engineering And Technology

Computer Engineering

Subject : Artificial Intelligence and Machine Learning in Healthcare (AIML in Healthcare)

Project Report On

Symptom-to-Report Analyzer (CareNote)

Submitted By

Ayush Mandal - Roll No. 33

Swayam Shah - Roll No. 49

Vinayak Singh - Roll No. 56

Degree Program: B.E. Computer Engineering (Final Year)

Academic Year
2026–2027

1. Introduction

The Symptom-to-Report Analyzer, branded in the application as CareNote, is an educational web application developed to help a person organise the symptoms they have noticed into a clear, structured and shareable summary. A registered user selects symptoms from a searchable checklist, adds the duration, a self-reported severity level and optional notes, and the system produces a report preview and a downloadable PDF. The system is built using Python and the Flask web framework, with HTML, CSS, Bootstrap and Chart.js for the user interface, and SQLite for storing user accounts and saved reports.

A Random Forest classifier from scikit-learn is trained on a public disease–symptom dataset. The selected symptoms are converted into a multi-hot feature vector and the classifier returns the closest matching dataset label together with the dataset's description text. This label is presented strictly as a reference pattern from the dataset and not as a diagnosis, a probability or a treatment recommendation. Alongside the model, a separate rule-based warning-sign checklist screens for seven urgent symptoms (such as chest pain, breathing difficulty and sudden weakness). This checklist is deliberately independent of the machine-learning output so that a classifier error can never suppress an urgent warning.

The application provides user registration and login with hashed passwords, CSRF protection, a personal dashboard with charts, a searchable report history with edit, delete and PDF export, a profile page for changing the password, and a Model & Dataset page that openly reports dataset statistics and the limits of the evaluation. Before training, exact duplicate symptom patterns are removed so that the 4,920 raw rows reduce to 304 unique patterns, which prevents duplicated rows from leaking between the training and test sets.

Thus, the project combines a Flask web application, data pre-processing with Pandas, supervised machine learning with scikit-learn, rule-based safety screening, SQLite storage, visualisation and PDF generation into one integrated educational platform. It is developed as a coursework prototype and is not intended for medical diagnosis, triage or to replace professional medical advice.

2. Problem Statement

People often find it difficult to explain their symptoms clearly during a healthcare appointment. Details such as which symptoms appeared, how long they have lasted and how severe they feel are easily forgotten or described in a scattered way, so useful context may be lost. Notes kept on paper or in chat messages are rarely structured, and are difficult to search, compare or share.

A second problem is the safe use of machine learning in health-related software. Public symptom datasets are convenient for building a classifier, but they contain many repeated rows, a very small number of independent examples per disease, and unknown clinical validity. A naive project that reports a high accuracy figure, or presents a predicted disease as a diagnosis, can mislead users and give a false sense of reassurance or alarm. Urgent warning signs must also never depend on a model that can be wrong.

Therefore, there is a need for a Symptom-to-Report Analyzer that can collect symptom information in a structured form, apply a transparent machine-learning workflow with duplicate removal and

honest evaluation, run an independent safety checklist, store reports per user, and export a readable PDF – while clearly stating that it is an educational prototype and not a diagnostic tool.

3. Objective

The principal objectives of the project are:

1. To build a responsive Flask web interface that lets a registered user select symptoms from a searchable list and enter duration, self-reported severity and optional notes.
2. To load, clean and normalise symptom labels from the disease–symptom CSV files into stable lowercase underscore keys using Pandas.
3. To identify and remove exact duplicate disease/symptom patterns before the train/test split so that evaluation is not inflated by data leakage.
4. To encode multi-symptom records as binary (multi-hot) feature vectors and train a Random Forest classifier that returns a reference dataset pattern for the selected symptoms.
5. To add an independent, conservative rule-based warning-sign checklist whose result is never controlled by the machine-learning model.
6. To implement secure user management with registration, login, password hashing, CSRF protection, session handling, password change and an email-based password-reset flow.
7. To store each user's reports in SQLite, restrict access to the owner, and provide a dashboard with charts, a searchable report history, and edit, delete and PDF-export functions using ReportLab.
8. To demonstrate responsible healthcare AI by reporting dataset limitations, qualifying the evaluation score, avoiding clinical claims, and showing a clear safety disclaimer on every relevant page and in the generated PDF.

4. Project Description

The Symptom-to-Report Analyzer (CareNote) is a web-based educational application that combines a Flask backend, a scikit-learn Random Forest model, a rule-based safety checklist, SQLite storage and ReportLab PDF generation. Registered users choose symptoms and add context, and the system produces a structured report preview showing the reported information, the independent safety message, the dataset reference pattern with its description, and general next-step guidance. Reports are saved to the user's account and can be searched, edited, deleted and exported as PDF. The application is intended for educational purposes and is not a clinical diagnostic tool.

Project Details

Item	Details
Data Source	Public disease–symptom dataset (Kaggle) supplied as four CSV files in the data/ folder; user-selected symptoms entered through the web form
Data Size	dataset.csv: 4,920 rows, 41 disease labels, 17 symptom columns; 304 unique patterns after deduplication; 131 distinct symptom keys
Input Data	Selected symptoms (137 options in the form, including 7 warning-sign options), duration (up to 100 characters), severity (Mild / Moderate / Severe / Unsure) and notes (up to 1,200 characters)
Backend	Python 3 with the Flask web framework, Flask-WTF (CSRF protection), Werkzeug (password hashing) and itsdangerous (signed reset tokens)
Frontend	Server-rendered Jinja2 templates with Bootstrap 5, custom CSS, a small JavaScript file and Chart.js for dashboard charts
Database	SQLite (instance/users.sqlite3) with a users table and a reports table; report content stored as JSON
Data Processing	Pandas for loading, normalising and deduplicating symptom records; NumPy for array handling
Machine Learning	scikit-learn MultiLabelBinarizer for multi-hot encoding and RandomForestClassifier (200 trees for evaluation, 300 trees for serving, class_weight = balanced_subsample)
Safety Screening	Independent rule-based checklist of 7 urgent symptoms, not controlled by the ML model
Report Generation	Downloadable PDF symptom summary generated with ReportLab
Visualisation	Doughnut chart of reported severity and bar chart of most frequent symptoms on the dashboard (Chart.js)
Security and Authentication	Registration and login, minimum 10-character password, hashed passwords, CSRF tokens, HttpOnly and SameSite session cookies, owner-only report access, signed 30-minute reset tokens, 32 KB request-size limit
Testing	pytest suite with 8 automated tests covering data, routes, safety rules and PDF output
Safety Disclaimer	States that the application is an educational prototype and must not be used for diagnosis, triage or treatment decisions

Note on Execution

The application runs locally with the Flask development server on http://127.0.0.1:5000. The dataset CSV files must be present in the data/ folder. When the application starts, the model is built once and cached: the data is loaded and deduplicated, a stratified hold-out split is evaluated, and the serving Random Forest is then fitted on all unique patterns. Because the server is a development server, the project demonstrates an educational prototype rather than a production deployment. Before any real deployment, a strong FLASK_SECRET_KEY must be set, HTTPS must be used with SESSION_COOKIE_SECURE enabled, and a production WSGI server must replace the development server.

The classifier output is a dataset label chosen from the labels it has seen. It depends on the symptoms selected and on the small, deduplicated training data, and is not a medically validated diagnosis. Password-reset emails are sent only when SMTP settings (MAIL_HOST, MAIL_PORT, MAIL_USERNAME, MAIL_PASSWORD, MAIL_FROM) are configured; otherwise signed-in users can change their password from the Profile page.

Description of Data Records

The application stores two kinds of records in SQLite. The users table keeps account details, and the reports table keeps each saved report, with the full report content stored as a JSON document.

Field	Meaning	Valid Range / Format
users.id	Unique identifier of a user	Integer / auto-increment
users.email	Login email of the user	Unique, case-insensitive, up to 254 characters
users.password_hash	Salted hash of the user's password	Werkzeug hash string
users.created_at	Account creation time	ISO-8601 UTC timestamp
reports.id	Unique identifier of a saved report	Integer / auto-increment
reports.user_id	Owner of the report	Foreign key to users.id (cascade delete)
reports.title	Display title of the report	Text, up to 100 characters
reports.report_json	Full report content (see below)	JSON text
reports.created_at / updated_at	Creation and last-edit times	ISO-8601 UTC timestamps
selected_keys / symptoms	Selected symptom keys and display labels	List of strings
duration	How long the symptoms have lasted	Text, up to 100 characters
severity	Self-reported severity	Mild, Moderate, Severe or Unsure
notes	Optional free-text notes	Text, up to 1,200 characters
analysis	Reference pattern, description, matched features, data source and disclaimer	JSON object
warning	Result of the independent safety checklist (urgent, flagged, message)	JSON object

Application Routes

Route	Method	Purpose
/	GET	Landing page with product overview and safety notice
/register, /login, /logout	GET, POST	Account creation, sign-in and sign-out
/forgot-password, /reset-password/<token>	GET, POST	Email-based password reset using signed, time-limited tokens
/dashboard	GET	Overview with report count, recent reports and charts
/new-report, /analyze	GET, POST	Symptom form and report generation

Route	Method	Purpose
/reports	GET	Report history with search
/reports/<id>	GET	Report preview for an owned report
/reports/<id>/edit, /reports/<id>/delete	GET, POST	Edit or delete a saved report
/reports/<id>/export.pdf, /report.pdf	GET, POST	PDF export of a saved report or direct PDF export
/profile	GET, POST	Account details and password change
/about-model	GET	Dataset statistics and evaluation notes

Project Modules

Module (File)	Purpose
app.py	Application factory; routes, authentication, session handling, SQLite access, report preparation and PDF endpoints
modules/data_pipeline.py	Loads the CSV files, normalises symptom names, removes duplicate patterns, loads descriptions, precautions and severity weights, and builds the symptom list
modules/model.py	SymptomPatternModel: multi-hot encoding, stratified evaluation, Random Forest training and prediction
modules/safety_screening.py	Independent warning-sign checklist for seven urgent symptoms
modules/report_generator.py	Builds the PDF symptom summary using ReportLab
train_model.py	Runs the evaluation and saves reports/metrics.json
inspect_dataset.py	Prints dataset checks, class counts and most frequent symptoms
templates/*.html	landing, auth, index (form), result, dashboard, history, profile and model_info pages
static/style.css, static/app.js	Styling, entrance animation, form-submit feedback and delete confirmation
tests/test_app.py	Automated pytest tests
data/*.csv	dataset.csv, Symptom-severity.csv, symptom_Description.csv, symptom_precaution.csv
docs/	Project report notes, dataset review, viva questions and architecture diagram

5. Methodology / Proposed Approach

The proposed Symptom-to-Report Analyzer follows a systematic pipeline that combines data pre-processing, supervised machine learning, independent rule-based safety screening, database storage and report generation.

Proposed Pipeline:

User Login → Symptom Selection and Context Entry → Form Validation → Symptom Normalisation → (Random Forest Reference Pattern ‖ Independent Warning-Sign Check) → Report Preview → SQLite Storage → Dashboard and History → PDF Export

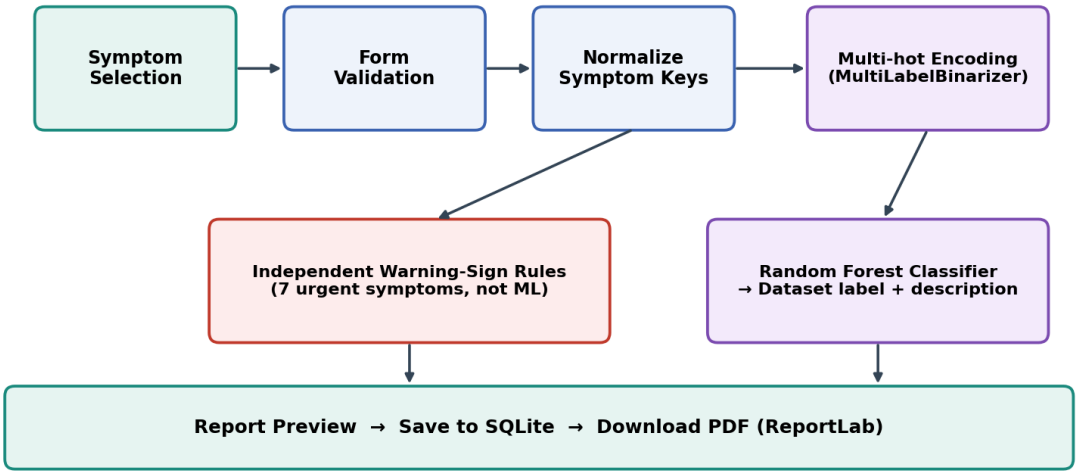


Figure 1: Processing Pipeline of the Symptom-to-Report Analyzer

Step 1: Dataset Loading and Inspection

The four CSV files are loaded with Pandas. The main file contains a disease label and up to 17 symptom columns per row. The inspect_dataset.py script reports the raw row count, the number of unique patterns, class counts and the most frequent symptoms so that the data can be checked before any model is trained.

Step 2: Symptom Normalisation

Each symptom value is converted to a stable key by trimming, lower-casing, replacing spaces and hyphens with underscores, removing other characters, and collapsing repeated underscores. Empty values such as nan, none, null and na are discarded. The symptoms of each row are then collected into a sorted set.

Step 3: Duplicate Removal

Rows that have the same disease label and the same symptom set are exact repeats. These are removed before the train/test split, reducing 4,920 raw rows to 304 unique patterns across 41 labels. This prevents the same pattern appearing in both training and test data and artificially inflating the evaluation score.

Step 4: Feature Encoding

The symptom sets are converted to binary vectors using MultiLabelBinarizer. Each of the 131 distinct symptoms becomes one column, with 1 meaning the symptom is present, giving a 304 × 131 feature matrix.

Step 5: Model Evaluation and Training

The deduplicated data is divided with a stratified 75/25 split (228 training and 76 test patterns). A Random Forest with 200 trees and balanced sub-sample class weights is evaluated on the test set using accuracy and a per-class precision, recall and F1 report. The serving model (300 trees) is then trained on all 304 unique patterns.

Step 6: User Authentication and Form Validation

The user registers with an email and a password of at least 10 characters, and signs in. Every protected route requires a valid session. On submission, only symptom keys that exist in the known list are accepted, at least one symptom is required, severity is restricted to four allowed values, and the duration and notes are length-limited.

Step 7: Independent Safety Screening

The selected symptoms are compared with a fixed set of seven urgent keys: chest pain, breathing difficulty, sudden weakness, confusion, severe bleeding, severe allergic reaction and fainting. If any is selected, an urgent warning is shown. If none is selected, the message clearly states that this does not rule out an emergency. This step runs separately from the classifier.

Step 8: Reference Pattern Generation

The selected symptoms that match the model's training features are encoded and passed to the Random Forest, which returns a dataset label. The matching description from symptom_Description.csv is displayed as reference text only. The model's probability scores are intentionally not shown as medical confidence, and the generic precautions file is not presented as personalised advice.

Step 9: Report Storage

The complete report (selected symptoms, duration, severity, notes, reference pattern and safety result) is stored as JSON in the reports table, linked to the user's id. All queries filter by the signed-in user, so one user cannot open another user's reports.

Step 10: Dashboard, History and Visualisation

The dashboard shows the number of saved reports, the most recent activity, a doughnut chart of reported severity and a bar chart of the five most frequently reported symptoms. The history page lists the user's reports with a search box and provides Open, Edit and Delete actions.

Step 11: PDF Report Generation

ReportLab builds a PDF containing the reported information, the independent safety screening message, the dataset reference pattern and its description, the training data source, and an important-health-information section with a privacy note.

Step 12: Security and Safety Measures

Passwords are stored only as salted hashes. Forms are protected by CSRF tokens, session cookies are HttpOnly and SameSite=Lax, redirect targets after login are restricted to local paths, and request bodies are limited to 32 KB. A disclaimer states that the tool is an educational prototype and must not replace professional medical advice, diagnosis or treatment.

6. Proposed System and Basic Concept

The Symptom-to-Report Analyzer integrates a Flask web application, a data pipeline, a scikit-learn Random Forest model, an independent safety checklist, a SQLite database and a ReportLab PDF

generator. The browser interface allows a registered user to select symptoms, view a structured report, browse history and download a PDF. The backend validates input, normalises symptoms, obtains a reference pattern from the model, runs the safety checklist and stores the result. It is an educational prototype, not a clinical diagnostic tool.

Basic Concept

Component	Role in the Project
Flask	Handles routing, form processing, sessions, authentication and template rendering
Jinja2 + Bootstrap 5	Provides the responsive landing, form, report, dashboard, history and profile pages
Chart.js	Draws the severity doughnut chart and the frequent-symptoms bar chart on the dashboard
SQLite	Stores user accounts and saved reports in a single local database file
Pandas / NumPy	Load, clean, normalise and deduplicate the CSV data and prepare arrays
MultiLabelBinarizer	Converts a set of symptoms into a multi-hot binary feature vector
Random Forest (scikit-learn)	Ensemble of decision trees that returns the closest dataset label for the selected symptoms
Safety Screening module	Rule-based checklist that flags seven urgent symptoms independently of the model
ReportLab	Generates the downloadable PDF symptom summary
Flask-WTF / Werkzeug / itsdangerous	CSRF protection, password hashing and signed password-reset tokens
pytest	Runs the automated tests for data, routes, safety rules and PDF output

Data Pre-processing and Classification Formulation

Each report is represented by the set S of symptoms selected by the user. The set is encoded into a binary vector x over the d = 131 symptoms known to the model:

$$x = [x_1, x_2, \ldots, x_d], \quad x_j = 1 \text{ if symptom } j \in S, \text{ otherwise } 0$$

The dataset is first reduced to its unique patterns, so that a (symptom set, label) pair appears only once:

$$D_unique = \{ (S, y) : \text{the pair } (S, y) \text{ occurs at least once in } D_raw \}$$

The Random Forest consists of T decision trees $h_1, h_2, \ldots, h_T$. The reference label for a vector x is the label receiving the most votes:

$$\hat{y} = mode \{ h_t(x) : t = 1, \ldots, T \}$$

The independent warning check is a simple set intersection with the urgent set U of seven symptoms; it does not use ŷ:

$$urgent = (S \cap U) \neq \emptyset$$

Parameter	Description
S	Set of symptoms selected by the user for one report
x	Multi-hot feature vector of length 131 created from S
D_raw / D_unique	Raw dataset (4,920 rows) and deduplicated dataset (304 unique patterns)
T	Number of trees in the forest (200 for evaluation, 300 for the serving model)
ŷ	Reference dataset label returned by the model (not a diagnosis)
U	Fixed set of seven urgent warning-sign symptoms

Only the information needed for the model is encoded; the model does not use age, medical history, onset pattern, examination findings or test results. A returned label is therefore only a pattern match inside a small educational dataset.

How It Works

1. User Authentication: The user registers or signs in. A valid session is required for the dashboard, report form, history, profile and PDF routes.

2. Symptom Entry: The user searches and ticks symptoms in the checklist and optionally enters a duration, a severity level and notes.

3. Validation and Normalisation: The backend accepts only known symptom keys, requires at least one symptom, and restricts severity and text lengths.

4. Reference Pattern: The model encodes the matching symptoms and returns a dataset label with its description and a non-diagnostic disclaimer.

5. Safety Screening: The independent checklist reports whether any of the seven warning signs was selected.

6. Storage: The report is saved to SQLite against the user's id, and the user is redirected to the report preview.

7. Dashboard and History: Saved reports feed the dashboard charts and the searchable report history, from which they can be edited, deleted or exported.

8. PDF Export: ReportLab creates a PDF summary that the user can download and share with a healthcare professional if they choose.

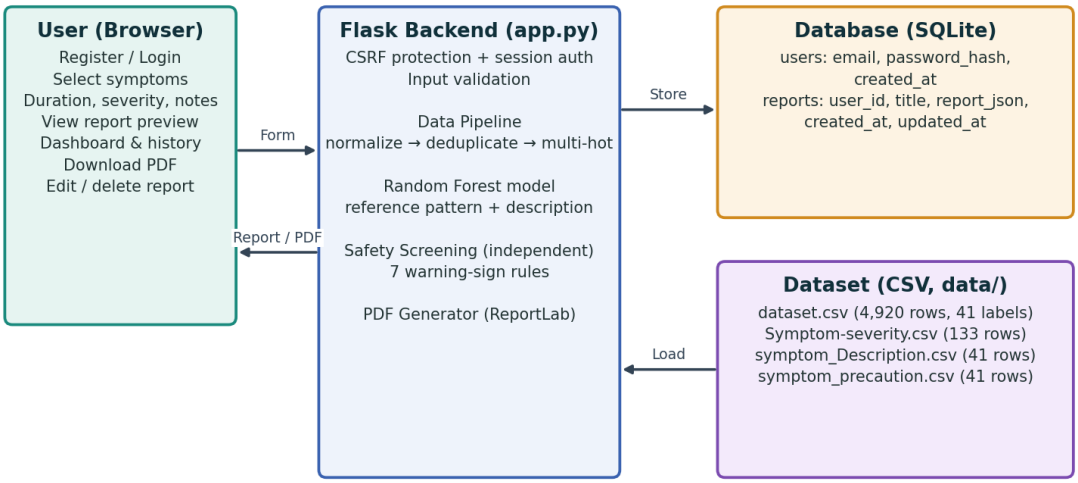


Figure 2: End-to-End Architecture of the Proposed System

Why It Is Suitable

- **Structured Symptom Capture:** A checklist with duration, severity and notes organises information that is otherwise scattered.
- **Transparent ML Workflow:** Normalisation, deduplication, encoding, stratified evaluation and training are all visible in code and explained in the interface.
- **Leakage Control:** Removing exact duplicate patterns before splitting gives a more honest evaluation than using raw rows.
- **Independent Safety:** Urgent warning signs are detected by fixed rules, so a classifier error cannot hide them.
- **User-specific Storage:** Reports are linked to a user account, and every query is filtered by the signed-in user.
- **Data Insight:** The dashboard charts show severity distribution and frequently reported symptoms across a user's saved reports.
- **Privacy-aware Design:** The form warns users not to enter identifying details, and the PDF includes a privacy note.
- **Cost-effective Development:** Flask, scikit-learn, Pandas, SQLite and ReportLab are open-source and need no licence fees.
- **Educational Safety:** Outputs are labelled informational and the application states that it is not a clinical diagnostic tool.

Alternative Approaches

Feature	This Project: Flask + Random Forest + Safety Rules	Plain Symptom Checklist (No ML)	ML-Only System Without Safety Rules
Symptom capture	Searchable checklist with duration, severity and notes	Checklist with notes	Checklist or free text
Pattern analysis	Random Forest reference pattern from deduplicated data	None	Model output only
Urgent warning detection	Independent rule-based checklist	Depends on implemented rules	Depends on the model, which can be wrong
Evaluation honesty	Duplicates removed before split; limits stated	Not applicable	Often evaluated on duplicated rows
Report output	Preview, history and PDF export	Depends on implementation	Depends on implementation
Risk of misleading user	Reduced through disclaimers and no probability display	Low	Higher if label shown as diagnosis

7. Implementation

The Symptom-to-Report Analyzer is implemented as a server-rendered Flask application. A Pandas-based data pipeline prepares the dataset, scikit-learn trains the classifier, a separate module performs the safety screening, SQLite stores users and reports, and ReportLab creates the PDF output. The implementation is divided into the software and tools used, followed by the important code sections.

Software and Tools Used

Software / Tool	Purpose
Python 3.10+	Backend development, data processing and model training
Flask 3.1	Web framework for routes, sessions and templates
Flask-WTF	CSRF protection for all forms
Pandas and NumPy	Data loading, cleaning, deduplication and array handling
scikit-learn 1.5+	MultiLabelBinarizer, RandomForestClassifier, train/test split and metrics
SQLite (sqlite3)	Storage of users and reports
ReportLab	PDF report generation
Bootstrap 5 and Chart.js	Responsive interface and dashboard charts
Matplotlib	Listed dependency for plotting support
pytest	Automated testing
VS Code	Code development and project management

Important Codes

1. Symptom Normalisation and Duplicate Removal

This code converts each symptom into a stable key and removes rows with the same disease label and symptom set before any train/test split.

```
def normalize_symptom(value):
    if pd.isna(value):
        return ""
    value = str(value).strip().lower()
    if value in {"", "nan", "none", "null", "na"}:
        return ""
    value = re.sub(r"[\s\-]+", "_", value)
    value = re.sub(r"[^a-z0-9_]", "", value)
    value = re.sub(r"_+", "_", value)
    return value.strip("_")

def load_clean_dataset(deduplicate=True):
    df = load_raw_dataset()
    before = len(df)
    if deduplicate:
        df = df.drop_duplicates(
            subset=["symptoms", "disease"]
        ).reset_index(drop=True)
    stats = {
        "raw_rows": before,
        "unique_patterns": int(len(df)),
        "removed_duplicate_rows": int(before - len(df)),
        "disease_classes": int(df["disease"].nunique()),
        "unique_symptoms": int(len({s for row in df["symptoms"]
                                    for s in row})),
    }
    return df, stats
```

2. Feature Encoding, Evaluation and Prediction

The model encodes symptom sets as multi-hot vectors, evaluates a Random Forest on a stratified hold-out split, and then trains the serving model on all unique patterns.

```
class SymptomPatternModel:
    def __init__(self):
        self.df, self.dataset_stats = load_clean_dataset(deduplicate=True)
        self.features = sorted({s for row in self.df["symptoms"]
                                for s in row})
        self.mlb = MultiLabelBinarizer(classes=self.features)
        self.X = self.mlb.fit_transform(self.df["symptoms"])
        self.y = self.df["disease"].to_numpy()
        self.model = RandomForestClassifier(
            n_estimators=300, random_state=42,
            class_weight="balanced_subsample", n_jobs=-1)
        self.evaluation = self._evaluate()
        self.model.fit(self.X, self.y)

    def _evaluate(self):
        X_train, X_test, y_train, y_test = train_test_split(
            self.X, self.y, test_size=0.25,
            random_state=42, stratify=self.y)
        evaluator = self._new_classifier()
        evaluator.fit(X_train, y_train)
        pred = evaluator.predict(X_test)
        return {"accuracy": float(accuracy_score(y_test, pred)), ...}

    def predict(self, selected):
        known = sorted(set(selected).intersection(self.features))
        vec = self.mlb.transform([known])
        label = str(self.model.predict(vec)[0])
        # predict_proba is not shown as a medical confidence score.
        return {"pattern": label,
                "description": self.descriptions.get(label, ""),
                "matched_features": known, "disclaimer": DISCLAIMER}
```

3. Independent Safety Screening

The warning-sign checklist is a separate module and does not use the model output.

```
URGENT_KEYS = {
    "chest_pain", "breathing_difficulty", "sudden_weakness",
    "confusion", "severe_bleeding",
    "severe_allergic_reaction", "fainting",
}

def check_warning_signs(selected):
    flagged = sorted(URGENT_KEYS.intersection(set(selected)))
    if flagged:
        return {"urgent": True, "flagged": flagged,
                "message": "One or more selected symptoms can be "
                           "warning signs of a serious condition. "
                           "Seek urgent medical assessment now. ..."}
    return {"urgent": False, "flagged": [],
            "message": "No warning sign from this prototype's limited "
                       "checklist was selected. This does not rule out "
                       "an emergency. ..."}
```

4. Database Tables

SQLite tables are created automatically when the application starts. Deleting a user cascades to that user's reports.

```
db.execute("""CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    email TEXT NOT NULL UNIQUE COLLATE NOCASE,
    password_hash TEXT NOT NULL,
    created_at TEXT NOT NULL)""")
db.execute("""CREATE TABLE IF NOT EXISTS reports (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    title TEXT NOT NULL DEFAULT 'Symptom summary',
    report_json TEXT NOT NULL,
    created_at TEXT NOT NULL,
    updated_at TEXT NOT NULL,
    FOREIGN KEY(user_id) REFERENCES users(id) ON DELETE CASCADE)""")
```

5. Registration with Password Hashing

Passwords must be at least 10 characters and are stored only as hashes.

```
@app.route("/register", methods=["GET", "POST"])
def register():
    if request.method == "POST":
        email = request.form.get("email", "").strip().lower()[:254]
        password = request.form.get("password", "")
        confirm = request.form.get("confirm_password", "")
        if "@" not in email or len(email) < 5:
            flash("Enter a valid email address.", "danger")
        elif len(password) < 10:
            flash("Use a password with at least 10 characters.", "danger")
        elif password != confirm:
            flash("The passwords do not match.", "danger")
        else:
            db.execute("INSERT INTO users (email, password_hash, "
                       "created_at) VALUES (?, ?, ?)",
                       (email, generate_password_hash(password), now))
```

6. Report Preparation and Analysis Route

The route validates the form, calls the model and the safety checklist, and saves the report for the signed-in user.

```
def prepare_report(form):
    all_options = {x["key"]: x["label"] for x in get_all_symptoms()}
    selected = sorted(set(x for x in form.getlist("symptoms")
                          if x in all_options))
    if not selected:
        return None, "Select at least one symptom to create a report."
    severity = form.get("severity", "unsure").strip().lower()
    if severity not in {"mild", "moderate", "severe", "unsure"}:
        severity = "unsure"
    analysis = get_model().predict(selected)
    warning = check_warning_signs(selected)
    return {"selected_keys": selected,
            "symptoms": [all_options[s] for s in selected],
            "duration": form.get("duration", "").strip()[:100]
                        or "Not provided",
            "severity": severity.title(),
            "notes": form.get("notes", "").strip()[:1200]
                     or "None provided",
            "analysis": analysis, "warning": warning}, None

@app.post("/analyze")
@login_required
def analyze():
    report, error = prepare_report(request.form)
    if error: ...   # re-render form with HTTP 400
    report_id = save_report(g.user["id"], report)
    return redirect(url_for("report_detail", report_id=report_id))
```

7. PDF Report Generation

ReportLab assembles the summary from the stored report and returns it as an in-memory PDF.

```
def build_pdf(report):
    buffer = BytesIO()
    doc = SimpleDocTemplate(buffer, pagesize=letter)
    styles = getSampleStyleSheet()
    story = [
        Paragraph("Symptom Summary Report", styles["ReportTitle"]),
        Paragraph("1. Reported information", styles["Heading2"]),
        Paragraph("<b>Selected symptoms:</b> " +
                  escape(", ".join(report["symptoms"])), styles["BodyText"]),
        Paragraph("2. Independent safety screening", styles["Heading2"]),
        Paragraph(escape(report["warning"]["message"]), styles["BodyText"]),
        Paragraph("3. Symptom pattern overview", styles["Heading2"]),
        Paragraph("<b>Dataset label:</b> " +
                  escape(report["analysis"]["pattern"]), styles["BodyText"]),
        ...
    ]
    doc.build(story)
    buffer.seek(0)
    return buffer
```

8. Output and Result

The following screens were captured from the running application using a test account and sample symptom entries. No real personal or medical data was used.

1. Registration and Login Pages

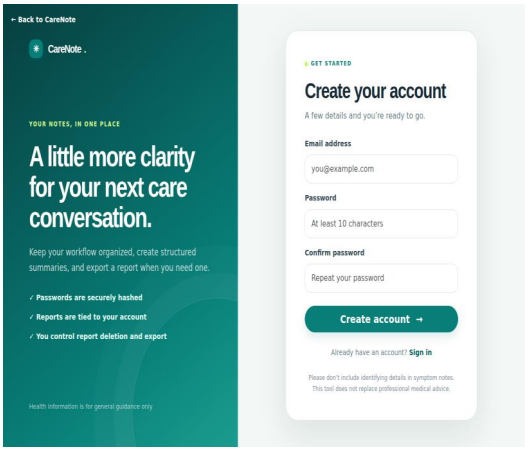


Figure 3: Registration page

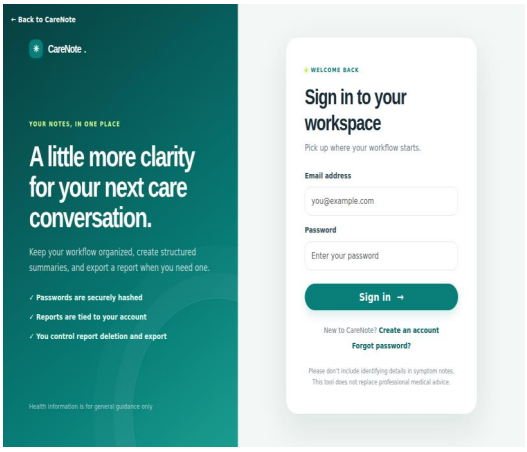


Figure 4: Login page

2. Landing Page and Dashboard

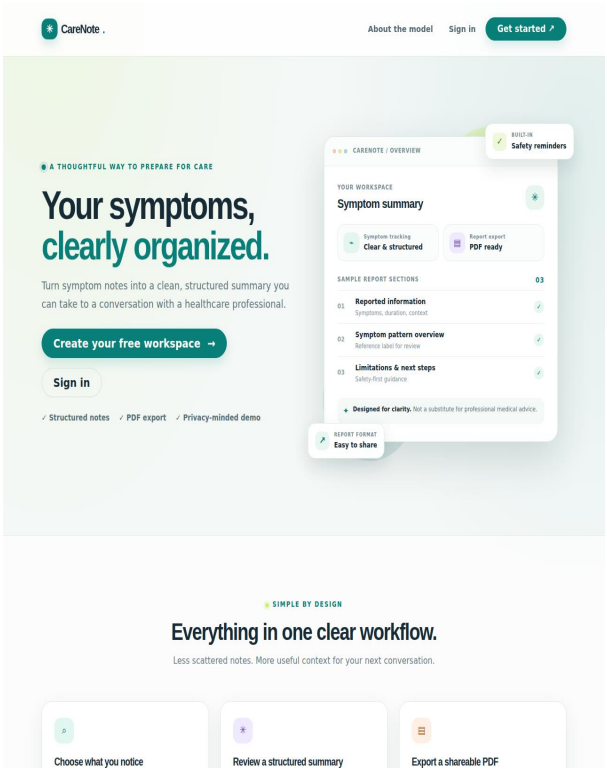


Figure 5: Landing page with workflow overview

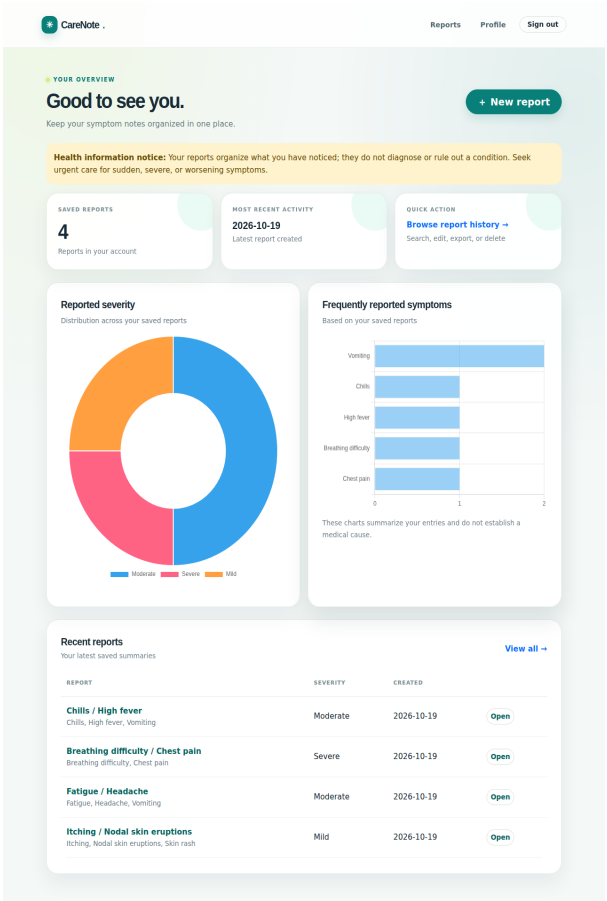


Figure 6: Dashboard with charts and recent reports

3. New Report Form

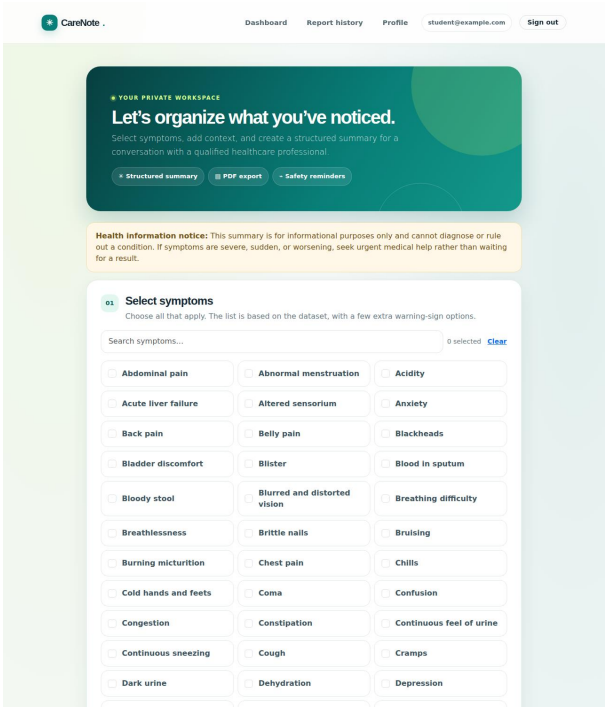


Figure 7: Searchable symptom checklist

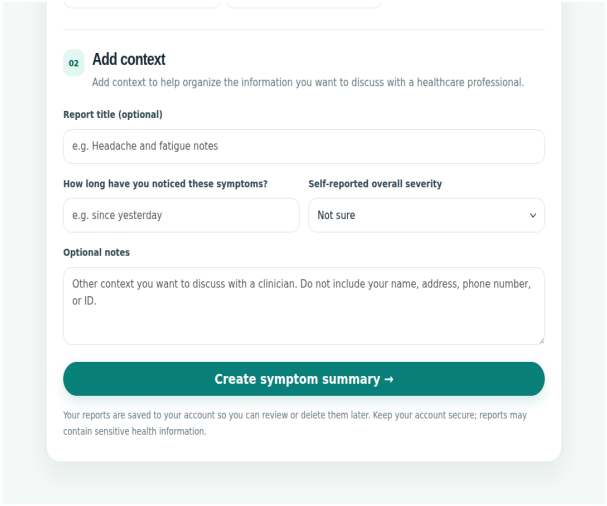


Figure 8: Duration, severity and notes fields

4. Report Preview and Urgent Warning

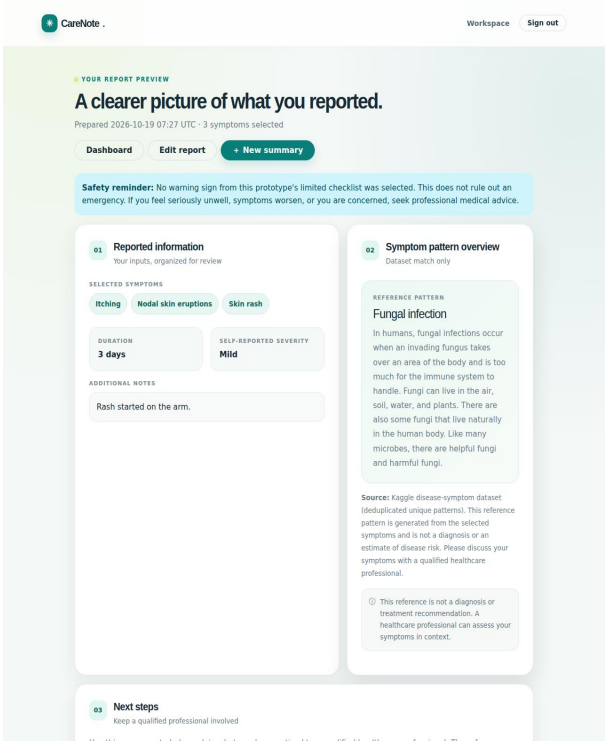


Figure 9: Normal report preview

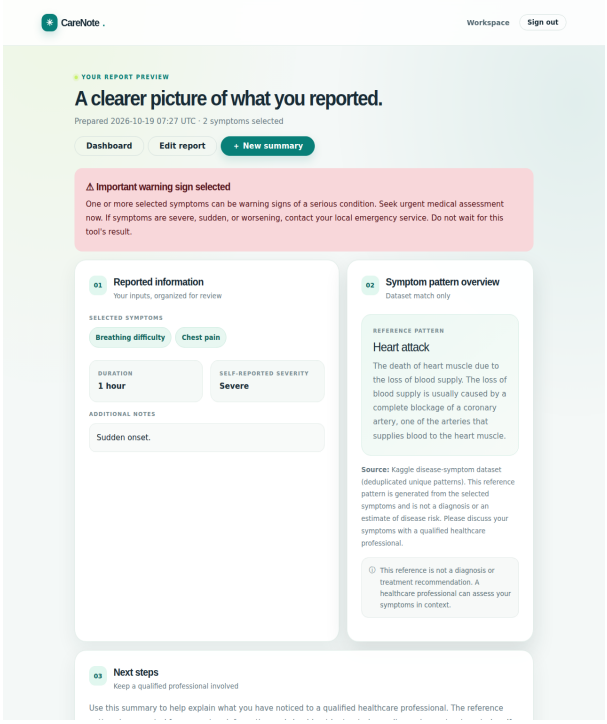


Figure 10: Urgent warning-sign alert

5. Report History and Profile

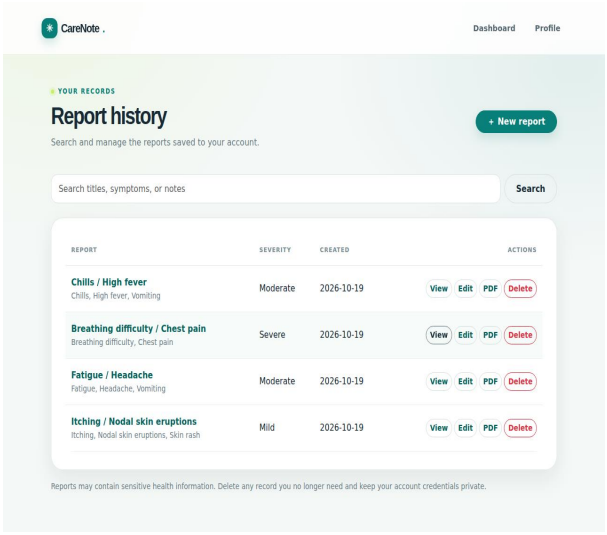


Figure 11: Report history with search

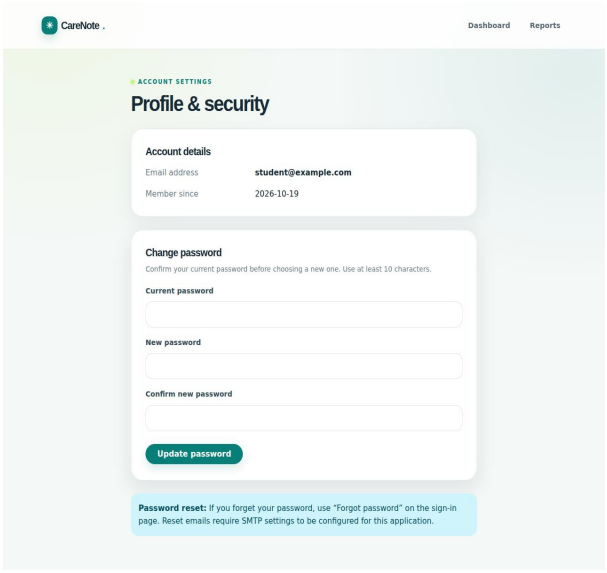


Figure 12: Profile and password change

6. Model Page and PDF Report

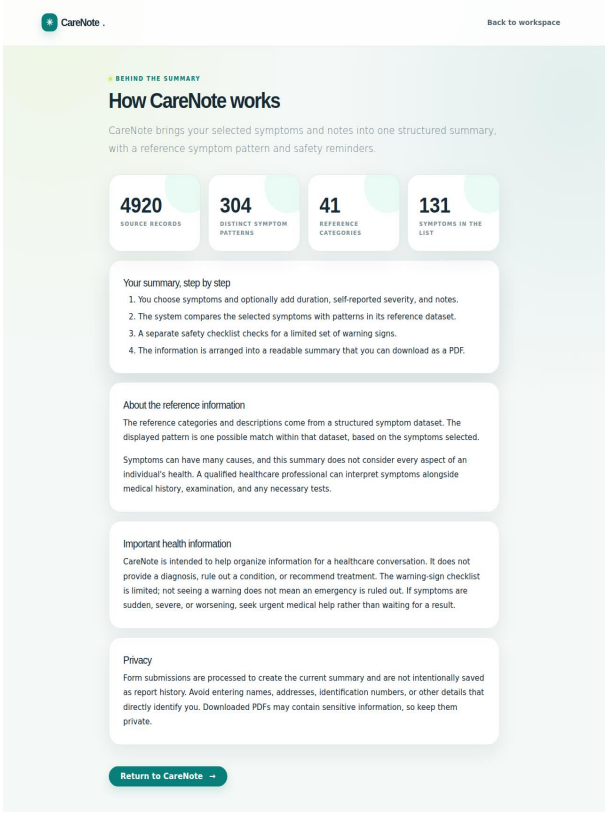


Figure 13: About-model page with dataset statistics

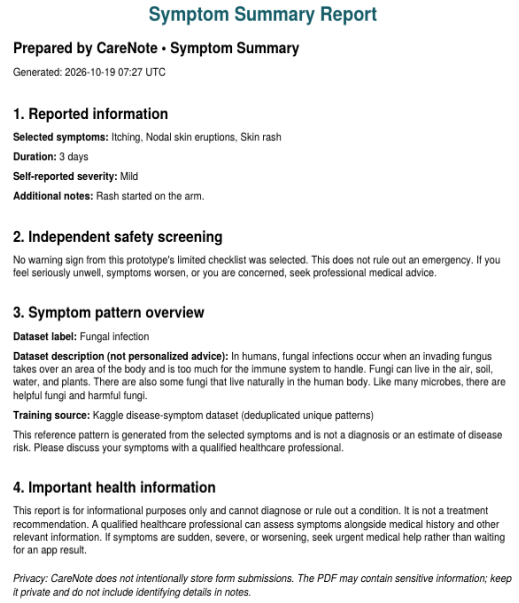


Figure 14: PDF summary generated by ReportLab

9. Test Summary

The project includes eight automated pytest tests in tests/test_app.py. All eight tests passed when run for this report (8 passed in about 6.6 seconds). The tests use a temporary SQLite database and a test configuration with CSRF checks disabled.

Test	Checks	Result
test_dataset_deduplicates_patterns	Raw rows equal 4,920, unique patterns are fewer than raw rows, and 41 disease classes remain	Passed
test_landing_page	Landing page loads and contains the brand name and the "does not diagnose" notice	Passed
test_register_and_sign_in	A new user can register, sign out and sign in again, reaching the dashboard	Passed
test_workspace_requires_login	Dashboard redirects unauthenticated users to the login page	Passed
test_empty_submission_rejected	Submitting no symptoms returns HTTP 400 with a clear message	Passed
test_urgent_warning_is_independent	Selecting breathing difficulty triggers the urgent warning without using the model	Passed
test_no_warning_is_not_reassurance	Absence of a warning still states that an emergency is not ruled out	Passed
test_pdf_response	PDF export returns HTTP 200, the application/pdf type and a valid %PDF header	Passed

Beyond the automated tests, the report screens in Section 8 were produced by manually registering a test user, creating four sample reports (including one containing warning-sign symptoms) and visiting every main page.

10. Performance Evaluation

A. Dataset and Model Evaluation Table

Parameter	Value	Remark
Raw dataset rows	4,920	41 diseases × 120 rows each
Unique patterns after deduplication	304	4,616 duplicate rows removed
Disease classes	41	About 7.4 unique patterns per class (min 5, max 10)
Distinct symptoms (features)	131	137 options in the form including warning signs
Train / test split	228 / 76	Stratified 75/25 split, random_state = 42
Algorithm	Random Forest	200 trees for evaluation, 300 for serving
Hold-out accuracy	100% (76/76)	Educational benchmark only; not clinical accuracy

Parameter	Value	Remark
Macro / weighted F1	1.00 / 1.00	Every class has a very small test support (1–2)
5-fold stratified CV: Random Forest	100%	Run on the deduplicated data
5-fold stratified CV: Logistic Regression	100%	Comparison model
5-fold stratified CV: Bernoulli Naive Bayes	97.7%	Comparison model

B. Additional Evaluation Considerations

- **Why the score is not clinical evidence:** Three different algorithms score close to 100% on the deduplicated data. This indicates that the labels are highly structured and that the dataset is simple and likely constructed, not that the model would perform equally on real patients.
- **Leakage control:** Deduplicating before the split removes the 4,616 repeated rows that would otherwise appear in both training and test sets.
- **Small class sizes:** With only about seven unique patterns per disease, a single misclassification changes a class score greatly, so per-class numbers are unstable.
- **Closed-set behaviour:** The classifier always returns one of the 41 labels it has seen; it cannot say that no label fits or recognise an unknown condition.
- **Safety rules are separate:** The seven-symptom warning checklist is incomplete and independent of model accuracy. Its absence of a warning must never be treated as reassurance.
- **Response behaviour:** The model is built once and cached, so later requests only encode the selected symptoms and run a forest prediction. PDF generation is in-memory.
- **Security limitations:** The development server, a local default secret key and SQLite are suitable for a demonstration only and need hardening before any real deployment.

11. Discussion

The Symptom-to-Report Analyzer shows how a small healthcare-related machine-learning workflow can be built responsibly. Flask provides the user interface and routes, Pandas prepares the data, scikit-learn trains a Random Forest, a separate module screens for warning signs, SQLite stores user reports, and ReportLab exports a readable summary.

A major lesson of the project is the importance of examining the data before trusting a model. The source file contains 4,920 rows, but only 304 are unique after normalisation and deduplication, and each disease has around seven independent patterns. Evaluation on the raw rows would give an exaggerated and misleading score. Even after deduplication the model reaches near-perfect scores with several algorithms, so the application reports this figure with a clear qualification instead of presenting it as medical accuracy.

The design also separates what the model is good at from what it must not be trusted with. The Random Forest supplies a reference pattern for educational display, but it is not used to decide whether a situation is urgent. That decision is made by a fixed set of rules that is easy to read and test. The interface avoids probability scores, avoids personalised treatment advice from the generic precaution file, and repeats the non-diagnostic disclaimer in the report preview and the PDF.

Limitations: The dataset's provenance and clinical validity are not established; symptom presence alone ignores age, history, onset, examination and tests; the warning checklist covers only seven symptoms; the model is closed-set; and the application has not been clinically validated or security audited.

12. Conclusion

The Symptom-to-Report Analyzer (CareNote) demonstrates an end-to-end AIML coursework workflow: dataset inspection, normalisation, duplicate removal, multi-hot feature encoding, Random Forest classification, honest evaluation, an independent safety checklist, a Flask web interface with authentication, SQLite report storage, dashboard charts, PDF reporting and automated testing.

The application helps users prepare organised symptom summaries for a conversation with a healthcare professional, while being explicit that its outputs are educational dataset matches and not medical advice, diagnosis or treatment. Further data validation, clinical review and security hardening would be required before any real-world use.

13. Future Scope

The system can be improved in the following ways:

- Replace or supplement the dataset with larger, independently sourced and clinically reviewed data, and re-evaluate with grouped cross-validation.
- Compare Random Forest with Logistic Regression, Naive Bayes and gradient-boosting models, and report confusion matrices and class-wise recall.
- Expand the warning-sign checklist with clinical guidance and add age, sex, history and symptom-onset inputs.
- Add free-text symptom entry using NLP and support for multiple languages.
- Move from SQLite and the Flask development server to a production database and WSGI server with HTTPS, rate limiting and audit logging.
- Add role-based access so that a clinician can view a report shared by the patient.
- Encrypt stored reports and add an option to export or delete all personal data.
- Add dashboard trends over time and a comparison between reports.
- Expand automated tests for authentication edge cases, report editing, deletion and PDF content.

14. References

• Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32. https://doi.org/10.1023/A:1010933404324

• Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

• Kaggle. Disease–symptom dataset (dataset.csv, Symptom-severity.csv, symptom_Description.csv, symptom_precaution.csv). https://www.kaggle.com/

• Pallets Projects. Flask Documentation. https://flask.palletsprojects.com/

• Flask-WTF. Flask-WTF Documentation (CSRF Protection). https://flask-wtf.readthedocs.io/

• Pallets Projects. Werkzeug Documentation. https://werkzeug.palletsprojects.com/

• scikit-learn. User Guide – Random Forests and Model Evaluation. https://scikit-learn.org/stable/

• pandas. pandas Documentation. https://pandas.pydata.org/docs/

• SQLite. SQLite Documentation. https://www.sqlite.org/docs.html

• ReportLab. ReportLab PDF Library User Guide. https://docs.reportlab.com/

• Chart.js. Chart.js Documentation. https://www.chartjs.org/docs/

• Bootstrap. Bootstrap 5.3 Documentation. https://getbootstrap.com/docs/5.3/

• OWASP Foundation. OWASP Top Ten. https://owasp.org/www-project-top-ten/